

Day 4 Stuck Sheet: "If you see this, do this"

Before you call your instructor, try the fix on this page. Most problems today have a small, known fix.

First, try these every time:

1. Save the file (**Ctrl + S**).
 2. Read the **red message** in the terminal. It usually names the file and the line number.
 3. Check that your terminal is inside the `notes-board` folder. Type `dir` and look for `server.js`.
 4. Compare your code with the lab sheet, one line at a time.
-

Part A: The database (SQLite)

Problem 1: `Cannot find module 'node:sqlite'`

Why it happens: `node:sqlite` is built into Node.js 24. An older Node.js may not have it.

Fix:

1. Type `node -v`. Does it start with `v24`?
 2. If not, you have a different version of Node.js. Tell your instructor. Install the Node.js 24 LTS file from the workshop folder, then close VS Code, open it again, and open a new terminal.
 3. If it does start with `v24` and the error stays, tell your instructor. This is **not tested on Windows yet**, and the instructor has a back-up plan.
-

Problem 2: The terminal shows an `ExperimentalWarning` about SQLite

Why it happens: Some versions of Node.js print an `ExperimentalWarning` when you use `node:sqlite`. (We saw it on Node.js 22.22.0 on Linux. The exact words may differ.) We saw no warning on Node.js 24.21.0 on Linux. It is not tested on Windows.

Fix: A warning is not an error. If the server then prints `Notes Board is running`, you can go on. Tell your instructor which `node -v` version you have. Do not try to hide the message by editing code.

Problem 3: database is locked

Why it happens: Two programs have the same database file open and are using it at once. Usually an **old server is still running** (in another terminal tab), or `try.js` from Part 1 is still running.

Fix:

1. Look at all the terminal tabs in VS Code. Press **Ctrl + C** in every one that runs a server.
 2. Run `node server.js` again.
 3. Still the same? Close VS Code completely, open it again, and try once more.
-

Problem 4: Windows says the file is in use, or I cannot delete `notes.sqlite` or `try.sqlite`

Why it happens: A running program still has the file open.

Fix: Press **Ctrl + C** in the terminal to stop the server (or the script). Then delete the file again.

If you want to **start with fresh notes**: stop the server, delete `notes.sqlite`, and start the server. It makes a new file with the 3 starter notes.

Problem 5: unable to open database file

Why it happens: SQLite could not make or open the file. The usual cause is the wrong folder, or a wrong value of `DATABASE_FILE` in `.env`.

Fix:

1. Type `dir`. Do you see `server.js`? If not, use `cd` to go into `notes-board`.
 2. Open `.env`. If it has a line that starts with `DATABASE_FILE=`, put `#` at the start of that line or delete the line.
 3. Run `node server.js` again.
-

Problem 6: My notes go back to the three starter notes after a restart

Why it happens (one of two reasons):

- You started the server from a **different folder**. The file name `notes.sqlite` has no folder, so SQLite made a **new** file in the folder where you ran the command. Your notes are in the other file.
- You deleted **all** notes. When the table is empty, `init()` adds the 3 starter notes again. This is how the code works.

Fix: Always run `node server.js` from inside `notes-board`. Look for `notes.sqlite` in that folder. If you find an extra `notes.sqlite` in another folder, you can delete it after you stop the server.

Problem 7: `TypeError: init is not a function`

Why it happens: `server.js` asks `data.js` for `init`, but `data.js` does not share it.

Fix:

1. Open `data.js` and go to the **last line**. It must be exactly:

```
module.exports = { init, listNotes, addNote, deleteNote };
```

2. Check that there is a function named `init` in the file (`async function init() { }`).
 3. Save both files and run `node server.js` again.
-

Problem 8: `no such table: notes`, or `no such column`

Why it happens: The table was not made, or it was made earlier with different column names (for example from a mistake in the first try).

Fix:

1. Compare the `CREATE TABLE` in `data.js` with the lab sheet. Names must match exactly: `id`, `title`, `text`, `created_at`.
 2. Press **Ctrl + C** to stop the server, **delete** `notes.sqlite`, and run `node server.js` again. The server makes a fresh table.
-

Problem 9: `SyntaxError` in `data.js`, or a red underline in VS Code

Why it happens: Something is missing: a `)`, a `}`, a quote or a backtick.

Fix:

1. Read the line number in the error.
 2. Check that line and the line above it.
 3. In the `init()` function, the SQL starts and ends with a **backtick** (```). Both backticks must be there.
 4. Click next to a `{` or `(`. VS Code highlights its partner. If nothing is highlighted, the partner is missing.
-

Problem 10: The page loads but the notes list is empty, or Failed to load notes

Fix:

1. Look at the terminal where the server runs. Is there a red error? Fix that first.
 2. Open `http://localhost:3000/api/notes` in the browser. Do you see a list of notes in text form? If you see an error there, the problem is in `data.js` or `server.js`.
 3. If the list is `[]`, the table is empty. Stop the server and start it again. `init()` adds the starter notes when the table is empty.
-

Part B: Git

Problem 11: 'git' is not recognized as an internal or external command

Why it happens: The terminal started before Git was installed, or Git was not installed.

Fix:

1. Close VS Code **completely**, open it again, and open a **new** terminal.
 2. Type `git --version`.
 3. Still the same? Restart the computer once, then try again.
 4. Still the same? Tell your instructor. Git may need to be installed again (see the Day 2 Lab Sheet, Part 2).
-

Problem 12: fatal: not a git repository (or any of the parent directories): .git

Why it happens: Git does not track this folder yet, or you are in the wrong folder.

Fix:

1. Type `dir`. Do you see `server.js`? If not, use `cd` to go into `notes-board`.
 2. If you are in `notes-board`, type `git init` once. Then try your command again.
-

Problem 13: nothing to commit, working tree clean

Why it happens: Git has no new changes to save. Either you already committed everything (this is good), or you did not change any file since your last commit.

Fix: Nothing is wrong. Type `git log --oneline`. If you see your commit, you can go on. If you changed a file and still see this message, save the file with **Ctrl + S** and try again.

Problem 14: Author identity unknown. *** Please tell me who you are.

Fix: Tell Git your name and email, then commit again:

```
git config user.name "Your Name"
git config user.email "you@example.com"
```

Problem 15: .env , node_modules or notes.sqlite shows up in git status

Why it happens: Git is not ignoring them. The usual causes:

- The `.gitignore` file is missing, or it is saved with a wrong name (for example `.gitignore.txt`).
- A line in `.gitignore` has a spelling mistake.
- You ran `git init` in the wrong folder (for example in `Workshop` , not in `notes-board`).

Fix:

1. Do not run `git add .` yet.
2. Type `dir` and look for `.gitignore` . Open it in VS Code. It must contain these lines:

```
node_modules/
.env
*.sqlite
*.sqlite3
*.db
```

3. Fix the file, save it, and type `git status` again.
4. Still wrong? Type `git status` and show your instructor.

If you already ran `git add .` but **have not committed**, type this to take a file out of the staging area. It does not delete the file from your computer:

```
git rm --cached .env
```

(Use the real name instead of `.env` if it is another file. For a folder, use `git rm -r --cached node_modules` .) Then check `git status` again.

Problem 16: I committed `.env` by mistake

Why it matters: Git keeps the history of every commit. If `.env` was committed, the password is in that commit. If you also pushed, it is on GitHub, and other people could read it. **Deleting the file in a new commit does not remove it from the history.** We cannot promise that it can be hidden again.

What to do:

1. **Tell your instructor now.**
2. **Change the password.** Write a new value after `NOTES_PASSWORD=` in `.env`, and stop using the old password anywhere.
3. Your instructor will help you fix the project. The easiest and safest way for a beginner is to make a **new** repository without the secret, and to stop using the old one. Your instructor will decide.
4. Fix `.gitignore` so that it does not happen again (see Problem 15).

This is a real-life lesson. Developers also do this by mistake. The habit that protects you is to run `git status` **before** `git add`.

Problem 17: A warning says `LF will be replaced by CRLF`

Why it happens: Windows and other systems end lines in a different way. Git is telling you that it may change them.

Fix: It is only a warning. It is commonly seen on Windows, but we have **not tested** it on the lab computers. If the commit works, go on.

Part C: GitHub

Problem 18: `fatal: remote origin already exists`

Why it happens: You already ran `git remote add origin ...` once.

Fix: See the address that is stored:

```
git remote -v
```

If it is **wrong**, replace it with the right one:

```
git remote set-url origin https://github.com/YOUR-USERNAME/notes-board.git
```

Problem 19: `fatal: repository '...' not found` , or the address has a wrong user name or a typo (wrong remote URL)

Why it happens: The address in `origin` is not the address of your repository.

Fix:

1. Run `git remote -v` and read the address.
 2. In the browser, open your repository on GitHub. Click the green **Code** button and copy the address.
 3. Compare: the user name and the repository name must be exactly the same, including the `.git` at the end.
 4. Set the right address with the `git remote set-url origin ...` command from Problem 18. Then run `git push -u origin main` again.
-

Problem 20: `error: src refspec main does not match any`

Why it happens: Git has no branch named `main` with a commit. Either you did not commit yet, or the branch has another name (for example `master`).

Fix:

1. Type `git log --oneline` . If it shows nothing, do the commit first (Lab Part 5).
 2. Type `git branch -M main` .
 3. Run `git push -u origin main` again.
-

Problem 21: `! [rejected] main -> main (fetch first)` OR `failed to push some refs`

Why it happens: The repository on GitHub is **not empty**. Often you ticked **Add a README file** when you created it, so GitHub has a commit that your computer does not have.

Fix: Ask your instructor. A simple way for a beginner is to **create a new empty repository** on GitHub (a new name, and no README), set the new address with `git remote set-url origin ...` (Problem 18), and push again. Do not use `git push --force` unless your instructor says so.

Problem 22: `Authentication failed` , `Permission denied` , `could not read Username` , or the browser window did not open (sign in fails)

Why it happens: GitHub did not accept your sign in. Git for Windows may open a browser window for this. **We have not tested this on the lab computers.** Other causes: you typed your normal GitHub password in the terminal (GitHub does not accept that for pushing), your account

email is not confirmed, or the lab network blocks the sign in page.

Fix:

1. Read the message in the terminal. Look for a window that opened **behind** VS Code.
 2. In the browser, check that you can sign in at `https://github.com`. If your email is not confirmed, GitHub asks you to confirm it. Do that, then run `git push -u origin main` again.
 3. If a browser window opens, finish the sign in there and allow Git.
 4. If it still fails, **raise your hand**. Your instructor will help.
 5. An alternative that your instructor can explain is a **personal access token**. This is a long text that works like a password for Git. **It is not tested for this workshop**. Never write a token in a file inside your project, and never commit it.
-

Problem 23: The push worked, but the page on GitHub is still empty

Fix:

1. Refresh the page with **F5**.
 2. Check that you are on the right repository (the user name and the name in the address bar).
 3. Run `git log --oneline` and `git remote -v`. Does the address match this repository?
 4. Is the **main** branch selected on the page? There is a branch menu above the file list.
-

Problem 24: `.env` is on GitHub

Fix: See Problem 16. **Tell your instructor now** and change the password.

Part D: Track B (PostgreSQL), optional

Problem 25: `Cannot find module 'pg'`

Fix: In the terminal, inside the folder where you run the server (`notes-board-pg`), run `npm install pg`.

Problem 26: `DATABASE_URL` is not set

Fix: Open `.env` in the same folder. Add the line `DATABASE_URL=postgresql://postgres:YOUR-PASSWORD@localhost:5432/notesboard` with your own password. Save and run the server again.

Problem 27: password authentication failed , Or connect ECONNREFUSED , Or database "notesboard" does not exist

Why it happens: The password in `DATABASE_URL` is wrong, the PostgreSQL server is not running, the port is different, or the database was not created.

Fix: Ask your instructor. Check the password and the port number against what you chose during the installation. Check that you ran `CREATE DATABASE notesboard;` . Track B is not tested on Windows, so some steps may be different on the real installer.

Other problems from earlier days

- **Error: listen EADDRINUSE: address already in use :::3000** : an old server is still running. Press **Ctrl + C** in its terminal. See the Day 2 Stuck Sheet.
 - **'node' is not recognized** : close VS Code, open it again, and open a new terminal.
 - **NOTES_PASSWORD is not set** : your `.env` file is missing or empty. Copy `.env.example` to `.env` and write a password after `NOTES_PASSWORD=` .
-

I am far behind

This is okay.

1. Copy the **done** folder for Day 4 from your instructor, and read it next to your own work.
2. The most important results of today are: the notes stay after a restart, and `.env` is **not** on GitHub.
3. The GitHub push can be finished at the start of Day 5.